МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки»

Лабораторная работа №6
по курсу «Программирование графических процессоров»

Моделирование и визуализация системы N взаимодействующих тел с использованием технологий OpenGL и CUDA (вариант 1)

Выполнил: Голев Андрей Михайлович
Группа: М8О-401Б-23
Преподаватель: А.Ю. Морозов

Москва, 2026

Условие

Цель работы - использование GPU для моделирования и визуализации системы N взаимодействующих тел: взаимодействие технологий CUDA и OpenGL (vbo + texture), решение проблемы коллизий множества объектов, создание простейшей «игры».

Сцена. Виртуальный куб со стороной 2a (a = 15), центр куба имеет координаты (0, 0, a): отрисовываются только нижняя грань (пол) и ребра. Внутри куба находятся N = 150 частиц (текстурированные сферы), которые условно считаются одноименно заряженными со стенками и друг с другом и потому отталкиваются друг от друга и от стенок куба; для частиц учитывается ускорение свободного падения. Нижняя грань закрашивается картой напряженности электрического поля, создаваемого частицами.

Игрок. Камера может свободно перемещаться по пространству без ограничений. При приближении камеры к частицам они «убегают» от нее - считается, что у игрока большой одноименный заряд q_c = 30. По нажатию кнопки мыши игрок совершает «выстрел» сильно заряженной частицей (q_b = 50) в направлении взгляда: пуля движется равномерно со скоростью v_b = 10, действует на другие частицы, а на нее саму ничто не влияет.

Математическая постановка. Обозначим через x_i, y_i, z_i положение i-й частицы, через v_x^i, v_y^i, v_z^i - ее скорость; l_ij - расстояние между частицами i и j, l_ic - между частицей и камерой, l_ib - между частицей и пулей. На каждом шаге (шаг интегрирования dt = 0.01) скорости пересчитываются по формуле: $v_x^i(k+1) = w*v_x^i(k) + K*q_i*[\sum_j q_j*(x_i - x_j)/(l_ij^3 + \varepsilon) + q_i*(x_i - a)/(|x_i - a|^3 + \varepsilon) + q_i*(x_i + a)/(|x_i + a|^3 + \varepsilon) + q_c*(x_i - x_c)/(l_ic^3 + \varepsilon) + q_b*(x_i - x_b)/(l_ib^3 + \varepsilon)]*dt$, где w = 0.99 - коэффициент замедления, K = 50 - коэффициент пропорциональности, $\varepsilon = 10^{-3}$ прибавляется в знаменатель, чтобы избежать деления на близкое к нулю значение. Слагаемые стенок входят только в свою компоненту: для оси x - отталкивание от плоскостей x = +-a, для оси y - от y = +-a, для оси z - от пола z = 0 и потолка z = 2a. В вертикальную скорость дополнительно входит −g*dt (g = 20 - «ускорение свободного падения»). Позиции обновляются как $x_i(k+1) = x_i(k) + v_x^i(k+1)*dt$ (аналогично y и z). Пуля движется равномерно: $x_b(k+1) = x_b(k) + v_b*dt$; в момент выстрела ее скорость совпадает с направлением объектива камеры: $v_x^b = v_b*\cos(yaw)*\cos(pitch)$, $v_y^b = v_b*\sin(yaw)*\cos(pitch)$, $v_z^b = v_b*\sin(pitch)$, а начальное положение немного смещено относительно камеры: $x_b(0) = x_c + v_x^b$. Начальные положения частиц: $x_i(0) = rand(-a, a)$, $y_i(0) = rand(-a, a)$, $z_i(0) = rand(0, 2a)$, скорости нулевые.

Карта напряженности. Текстура пола квадратная, n_p × n_p; переход от текселя (i, j) к координатам пола: $x = (2i/(n_p - 1) - 1)*a$, $y = (2j/(n_p - 1) - 1)*a$. Напряженность вычисляется по формуле $E = K*[\sum_p q_p/\sqrt{((x_p - x)^2 + (y_p - y)^2}$

+ (z_p − shift_z)² + ε) + q_b/√((x_b − x)² + (y_b − y)² + (z_b − shift_z)² + ε)], где shift_z = 0.75 - сдвиг для карты напряженности, чтобы картина была четче. Цвет пикселя (i, j) - (0, 0, min(E, 255)).

Программа должна компилироваться командой nvcc -Werror cross-execution-space-call -lm -lcublas -lcurand -lGL -lGLU -lglut -lGLEW. Все параметры (a, N, ε, w, K, dt, g, заряды, скорость пули, shift_z) заданы условием и подбирались бы экспериментально только ради красоты картинки.

Программное и аппаратное обеспечение

Код я писал, отлаживал и компилировал на виртуальной машине с GPU, на ней же запускал программу с разными сценариями и готовил кадры для отчета.

Графический процессор - NVIDIA GeForce GTX 1650: compute capability 7.5, 14 мультипроцессоров, глобальная память 4 086 169 600 байт (примерно 3,8 ГБ), разделяемая память на блок 48 КБ, константная память 64 КБ, 65 536 регистров на блок, максимум 1024 нити на блок, размер варпа 32.

Из софта: Ubuntu 24.04.1 LTS, драйвер NVIDIA 550.144.03, CUDA 12.4, компилятор nvcc 12.0 (V12.0.140). Сборку и прогоны сценариев делал скриптом на этой же машине, конвертацию кадров в PNG - скриптом на чистом Python без сторонних пакетов.

Ключевая особенность ВМ - у нее нет дисплея: отсутствуют X-сервер, Xvfb и библиотеки GLUT/GLU/GLEW, а заголовков EGL в системе нет (есть только libEGL.so.1 от glvnd и libEGL_nvidia.so.0 от драйвера). Поэтому стандартный путь «окно GLUT + интерактивное управление мышью и клавиатурой» невозможен, и архитектуру программы пришлось строить вокруг headless-рендеринга - как именно, описано в методе решения.

Метод решения

Ввод-вывод и управление. Так как на ВМ нет ни экрана, ни мыши, ни клавиатуры, интерактивное управление игроком заменено программным: все действия задаются во входной строке перед запуском. Программа читает из stdin одну строку - последовательность команд через точку с запятой, например: left,30;forward,20;up,10;yaw,25;pitch,8;shoot,1;time,10. Команды такие: left/right/forward/back/up/down,D - сдвиг камеры на D единиц (forward/back - по направлению взгляда, left/right - горизонтально перпендикулярно взгляду, up/down - вдоль оси z); yaw,deg - поворот камеры в плоскости xy параллельно полу; pitch,deg - наклон вверх; shoot,1 - выстрел по направлению взгляда; time,T - моделирование T секунд (T/dt шагов), таких команд может быть несколько - например, дать системе устояться, потом выстрелить и промоделировать еще. Команды выполняются последовательно, и после конца сценария программа один раз рисует кадр - вид из

камеры - и пишет его в stdout: 8 байт (ширина и высота, little-endian) и пиксели RGBA, 1024×768. Кадры конвертируются в PNG скриптом на Python. Такой ввод-вывод соответствует требованиям курса (данные из stdin, результат в stdout, никаких лишних файлов) и делает прогоны воспроизводимыми: один и тот же сценарий всегда дает один и тот же кадр, потому что начальные положения частиц генерируются rand() с фиксированным seed.

GL-контекст без X-сервера. Окно создать нельзя, поэтому GL-контекст создается через EGL напрямую на GPU-устройстве, без привязки к оконной системе. Библиотека libEGL.so.1 открывается через dlopen, нужные функции EGL берутся через dlsym с запасным поиском через eglGetProcAddress (заголовков EGL на ВМ нет, поэтому константы и прототипы описаны прямо в исходнике). Затем eglQueryDevicesEXT перечисляет EGL-устройства; программа перебирает их (первый проход - только устройства NVIDIA, второй - любые), получает display через eglGetPlatformDisplayEXT(EGL_PLATFORM_DEVICE_EXT, …), инициализирует его и создает контекст desktop OpenGL (compatibility profile). Конфиг подбирается перебором наборов атрибутов eglChooseConfig: с альфа-каналом и без, с глубиной и без и так далее - например, у NVIDIA на device-платформе eglChooseConfig не принимает атрибут EGL_ALPHA_SIZE, а полный набор с ним возвращает EGL_BAD_ATTRIBUTE. Поверхность - pbuffer, а если она не создается (на NVIDIA device-платформе eglCreatePbufferSurface вернула EGL_BAD_PARAMETER), используется surfaceless-контекст: eglMakeCurrent с EGL_NO_SURFACE. Рендер в любом случае идет не на поверхность, а в FBO, поэтому тип поверхности не важен.

Offscreen-рендер. При инициализации создается FBO: цветовой renderbuffer RGBA8 1024×768 и 24-битный глубинный renderbuffer. Сцена рисуется в FBO фиксированным конвейером: свои матрицы перспективы и lookAt (библиотеки GLU на ВМ нет), ребра куба - GL_LINES, пол - квад с текстурой карты напряженности, частицы - сферы собственным мешем (широта-долгота, 12×18 квадов) через display list, пуля - красная сфера поменьше; включен один направленный свет. Готовый кадр снимается glReadPixels из FBO (строки переворачиваются, потому что glReadPixels отдает их снизу вверх) и пишется в stdout.

Работа с текстурами. Текстур две. Текстура сферы (шахматная доска 64×64 RGBA) генерируется заранее скриптом на Python и встраивается прямо в исходник как C-массив - в рантайме программа не читает никаких файлов. Текстура карты напряженности - 512×512, и здесь основное взаимодействие CUDA и OpenGL: ядро fieldKernel считает E для каждого текселя (переход (i, j) → (x, y) и формула - из условия, цвет (0, 0, min(E, 255))) и пишет результат прямо в PBO (pixel buffer object), зарегистрированный в CUDA через cudaGraphicsGLRegisterBuffer. Перед запуском ядра PBO маппится (cudaGraphicsMapResources → cudaGraphicsResourceGetMappedPointer), ядро пишет по полученному указателю,

PBO анмапится, и текстура обновляется через glTexSubImage2D с привязанным GL_PIXEL_UNPACK_BUFFER - данные идут с GPU в текстуру, минуя хост.

VBO позиций частиц. Позиции частиц живут в VBO, тоже зарегистрированном через cudaGraphicsGLRegisterBuffer, и ядро физики пишет прямо в замапленный VBO. Так как формулы шага читают только позиции предыдущего шага (все слагаемые от x(k)), обновление делается ping-pong: буферы чередуются (VBO ↔ обычный буфер cudaMalloc), нечетные шаги пишут в VBO, четные читают из него, а после моделирования позиции копируются в обычный буфер. Если интероп недоступен - например, контекст оказался программным (Mesa/llvmpipe), с которым cudaGraphicsGLRegisterBuffer возвращает ошибку, - программа не падает, а переходит на запасной путь: обычные буферы cudaMalloc, cudaMemcpy и glTexSubImage2D из памяти хоста; физика в любом случае считается на GPU.

Физика. Ядро physicsKernel запускается с фиксированной конфигурацией grid(2)×block(128) - 256 нитей, по одной на частицу ($150 \le 256$), лишние нити сразу выходят. Каждая нить проходит по всем остальным частицам ($O(N^2)$, при $N = 150$ это дешево), считает отталкивание от частиц $q_j/(l_{ij}^3 + \varepsilon)$, слагаемые стенок по своей оси, отталкивание от камеры ($q_c = 30$) и пули ($q_b = 50$), гравитацию и замедление w; все вычисления в double. Пуля движется равномерно, обновляется на хосте и передается в ядро значением. Ядро карты поля запускается с фиксированной конфигурацией grid(32×32)×block(16×16) - 512×512 нитей, по одной на тексель, с grid-stride циклами на случай другого размера текстуры. Конфигурации нитей фиксированы и от входных данных не зависят - это требование курса.

Описание программы

Программа - один файл lab6.cu (около 1100 строк, из них ~16 КБ - встроенная текстура сферы).

Два ядра CUDA: physicsKernel(const double4 *src, double4 *dst, double4 *vel, int n, double3 cam, double3 bullet, int bulletActive) - шаг физики (читает src, пишет dst - ping-pong, vel обновляется на месте, камера и пуля передаются значением); fieldKernel(uchar4 *out, int np, const double4 *pos, int n, double3 bullet, int bulletActive) - карта напряженности с двумерным grid-stride циклом по текселям.

Остальные функции: initEgl - загрузка EGL через dlopen и создание контекста (два прохода по устройствам, перебор наборов атрибутов конфига, pbuffer или surfaceless); initGl - текстуры, VBO, PBO, FBO и display list сферы; initCuda - выбор устройства, генерация начальных положений частиц, регистрация VBO и PBO для интероп с запасным копирующим путем; simulateSteps - ping-pong запуск ядра физики и обновление пули; updateField - запуск ядра карты и обновление текстуры (интероп или копирование); renderFrame и writeOutput -

отрисовка кадра и вывод в stdout; parseScript - разбор входной строки; makePerspective, makeLookAt и buildSphereList - матрицы и меш сферы.

Обработка ошибок: все вызовы CUDA, EGL и GL проверяются; при любой ошибке печатается «ERROR: …» в stderr, и программа завершается с кодом возврата 0 (требование курса). В stderr также пишутся служебные строки - версия GL, выбранный тип поверхности, статус интероп: их удобно смотреть при запуске по ssh.

Результаты

Программа собрана на ВМ и запущена с четырьмя сценариями; каждый прогон выводил кадр в stdout, кадры сконвертированы в PNG. Рендер шел на аппаратном GL-контексте NVIDIA (surfaceless), CUDA-GL interop работал: ядро физики писало позиции прямо в VBO, ядро карты - прямо в PBO.

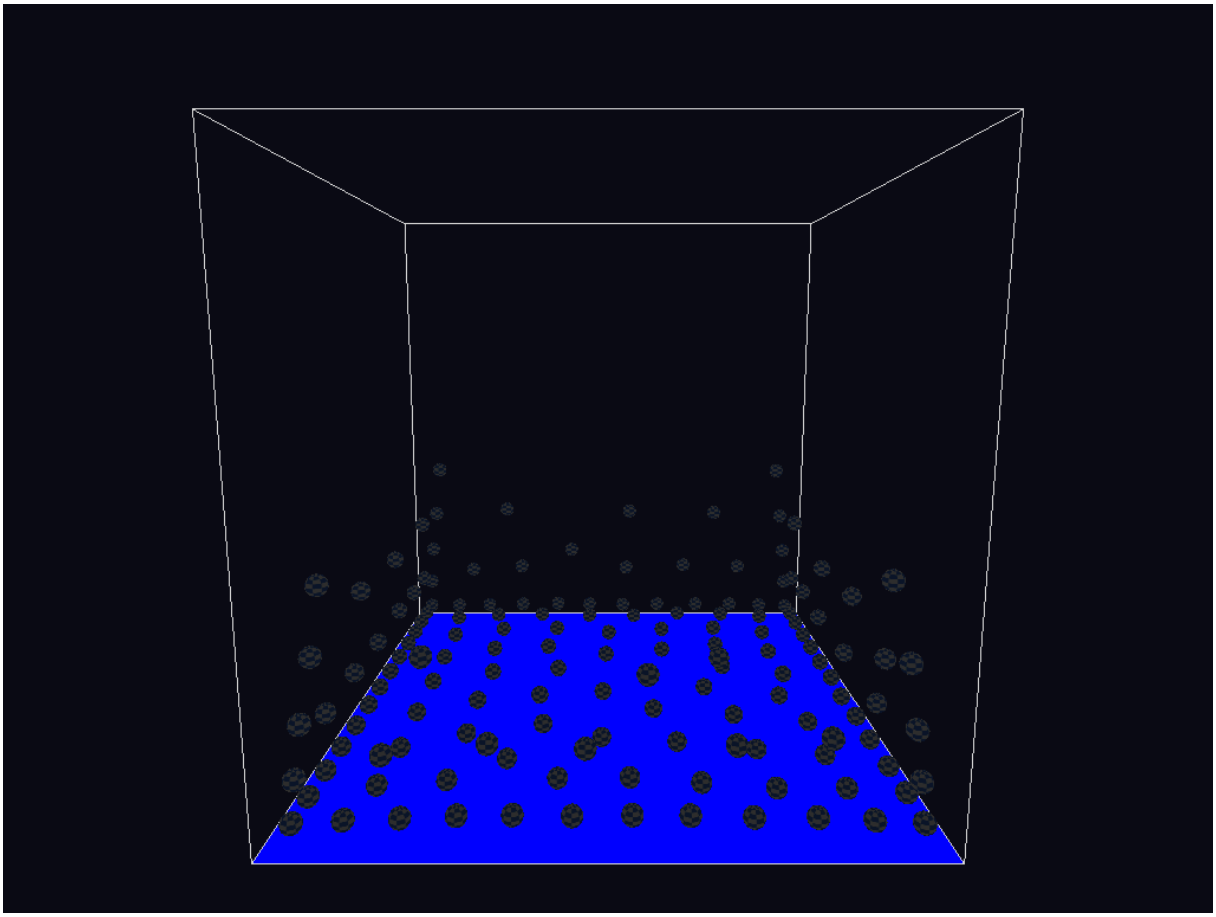


Рисунок 1 - Общий вид: система стабилизировалась (time,10), частицы осели на пол; на карте напряженности под ними синие пятна

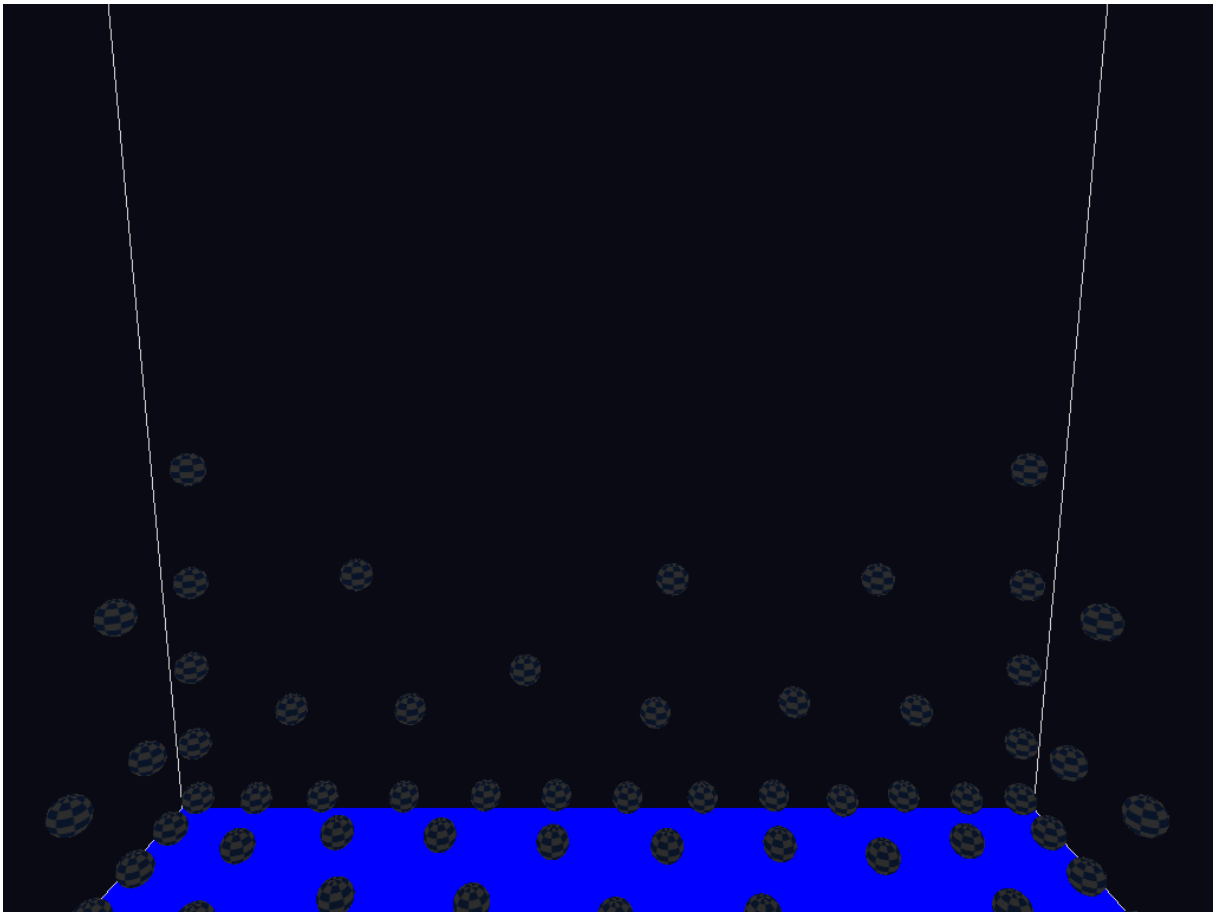


Рисунок 2 - Камера влетела внутрь куба (time,10;forward,35;time,5)

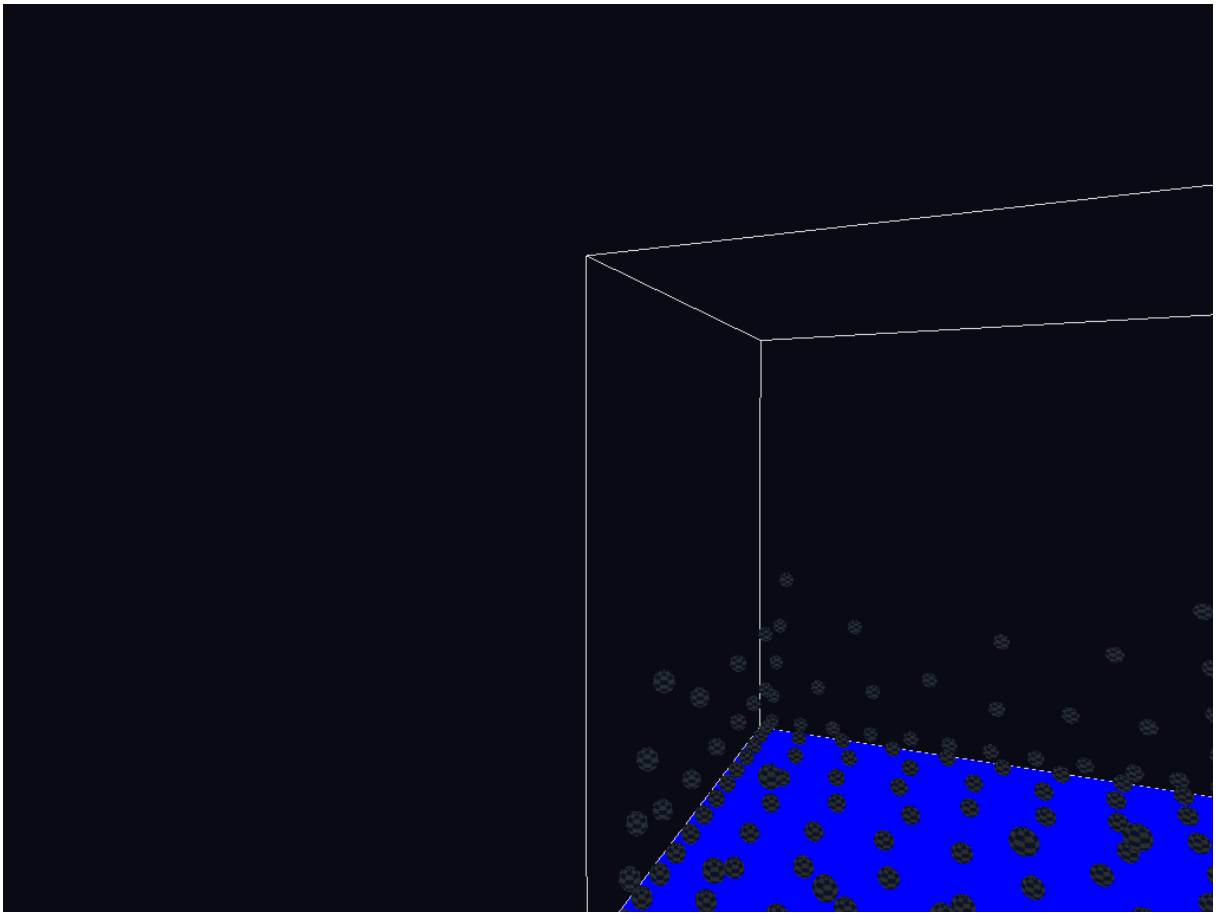


Рисунок 3 - Повернутая камера (time,10;yaw,25;pitch,8;time,1): поворот параллельно полу и наклон вверх, вид с другого ракурса

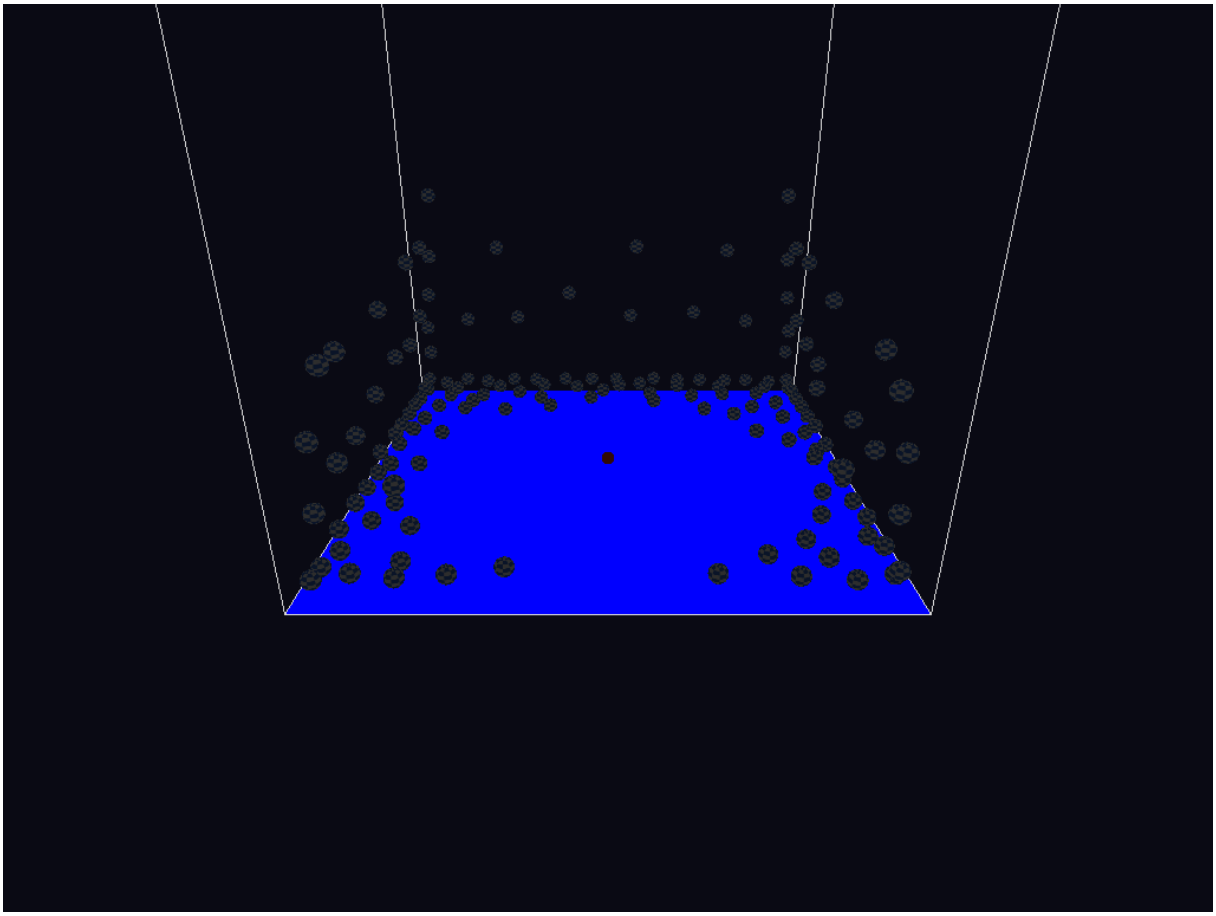


Рисунок 4 - Выстрел (time,10;pitch,−16;shoot,1;time,4): пуля (красная сфера) пришла в центр куба на уровне слоя частиц и расталкивает их; на карте поля - яркое пятно от заряда пули

Выводы

Главный технический результат - работающее взаимодействие CUDA и OpenGL: ядро физики пишет позиции частиц прямо в VBO, ядро карты напряженности - прямо в PBO, из которого обновляется текстура пола; данные не копируются через хост. Это и есть связка «vbo + texture» из задания.

Проблема отсутствия дисплея решена без суррогатов вроде виртуального X-сервера: GL-контекст создается через EGL на устройстве NVIDIA (headless), рендер идет в FBO, кадр снимается glReadPixels, а интерактивная игра заменена программным сценарием из stdin с выводом кадра в stdout. Тот же прием пригоден для серверного рендеринга и запуска графики в CI.

По пути пришлось обойти несколько практических особенностей: заголовков EGL в системе нет - функции загружаются через dlopen/dlsym и eglGetProcAddress; NVIDIA EGL на device-платформе не принимает EGL_ALPHA_SIZE в eglChooseConfig и не создает pbuffer - спасли перебор

конфигураций и surfaceless-контекст с рендером в FBO. Запасной путь (cudaMalloc + копирование через хост) делает программу работоспособной даже с программным GL-контекстом - физика в любом случае на GPU.